Basic (your first graph) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

Every LangGraph graph has the same 3 ingredients: a **state shape**, **nodes** (plain functions, state in, a small update out), and **edges** (what runs next). A `StateGraph` needs a real entry point and exit point too — `START` and `END`, both imported from `langgraph.graph`, are special markers, not nodes you write yourself.

The exact pieces:

- `from typing import TypedDict` and `class GraphState(TypedDict): message: str` — your state's shape, as a real type.
- `from langgraph.graph import StateGraph, START, END`
- `builder = StateGraph(GraphState)` — starts the builder, tied to your state shape.
- `def node_one(state: GraphState) -> dict: return {"message": ...}` — a node takes the whole state, returns only the keys it's changing (a **partial update**), not the whole state back.
- `builder.add_node("node_one", node_one)` and the same for `node_two`.
- `builder.add_edge(START, "node_one")`, `builder.add_edge("node_one", "node_two")`, `builder.add_edge("node_two", END)` — explicit start and end, not implied by node order.
- `graph = builder.compile()` then `graph.invoke({"message": "..."})`.

Think about what "a node returns a small update" actually means the moment your state has more than one field, or a field that's a list you want to *grow*, not replace. By default, LangGraph merges a node's returned dict into state one key at a time — for a plain field like `message: str`, that means the new value simply overwrites the old one. But what if two different nodes both need to append to the *same* running log, and you don't want node 2's write to erase node 1's?

The real design question isn't just "how do nodes update state" — it's "for each field, should a new node's update *replace* what's there, or *combine* with it?"

The extra piece that answers that question:

- **Reducers**, declared right in the state's type: `from typing import Annotated; import operator; log: Annotated[list[str], operator.add]`. Now, instead of overwriting `state["log"]` with whatever a node returns, LangGraph calls `operator.add(old_log, new_log)` — for lists, that's concatenation — so each node's returned `{"log": ["node_one ran"]}` *appends*, instead of replacing the whole history. Without a reducer, the default behavior is a plain overwrite, which is exactly right for a field like `message` but silently wrong for a field you meant to accumulate.

Hint 2 — The plan, and almost the whole thing

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

define:
    class GraphState(TypedDict):
        message: str

define:
    def node_one(state: GraphState) -> dict:
        return {"message": "Hello"}

define:
    def node_two(state: GraphState) -> dict:
        return {"message": state["message"] + ", World!"}

build:
    builder = StateGraph(GraphState)
    builder.add_node("node_one", node_one)
    builder.add_node("node_two", node_two)
    builder.add_edge(START, "node_one")
    builder.add_edge("node_one", "node_two")
    builder.add_edge("node_two", END)

run:
    graph = builder.compile()
    result = graph.invoke({"message": "start"})
    print(result)
```

Trace it by hand before running: what should `state["message"]` be right after `node_one` runs, and again right after `node_two`? Then run it and compare against the Solution.

Once that's working, add a field that needs to accumulate instead of overwrite:

```
add a second field, log, that should accumulate instead of overwrite:

class GraphState(TypedDict):
    message: str
    log: Annotated[list[str], operator.add]

each node now also returns {"log": ["node_one ran"]}
    or {"log": ["node_two ran"]}

run it and check: does state["log"] end up with BOTH entries, in order,
or does node_two's write erase node_one's?
```

```
# first_graph_practice.py
from typing import TypedDict, Annotated
import operator
from langgraph.graph import StateGraph, START, END

class GraphState(TypedDict):
    message: str
    log: Annotated[list[str], operator.add]

def node_one(state: GraphState) -> dict:
    return {"message": "Hello", "log": ["node_one ran"]}

def node_two(state: GraphState) -> dict:
    return {"message": state["message"] + ", World!", "log": ["node_two ran"]}
```

Wire this into a full graph yourself (same edges as before), invoke it with `{"message": "start", "log": []}`, and confirm `result["log"]` has both entries in order — then compare against the Solution.

Full solution: Show me the solution